

Can a Tiny AI on an 8 GB MacBook Write Real Business Logic?

Locating the spec-dialect boundary where a 3-billion-parameter coder model stops producing correct code

Muhammad Junaid — iteration, ladder experiments & repair loops · Muhammad Ansar — original research & problem · 2 July 2026

ABSTRACT

We ask whether Qwen2.5-Coder-3B-Instruct-4bit, run locally on Apple MLX, can generate a `process_calculations()` that scores a perfect 12/12 on a ten-rule pricing-and-commission specification. The model is fixed; the only lever is the *spec*. A re-dialected spec (v2) closes the gap from the original's 1/12 to **12/12 on the first greedy attempt**. We then build an eight-rung *spec ladder* — same ten rules and same targets, varying only the phrasing — to find the exact point of failure. The break is sharp and surprising: it is not the two constructs originally blamed (a multi-tier conditional and a truthy-string boolean), but the removal of the literal lookup tables, after which the 3B can no longer assemble the multi-step *commission* calculation. Three control rungs confirm it: keep the tables and the model tolerates every "trap." Finally, a white-box repair loop that feeds per-step intermediate diffs rescues the boolean rung to 12/12 at a low temperature, but does not reliably rescue the fully de-scaffolded rung — reaffirming that for a small model the dependable lever is the specification, not a cleverer repair loop. A second round then tests three escapes from that boundary. **Scale:** the identical ladder on the 7B leaves the greedy cliff exactly in place while making every rung reachable by sampling. **Hybrids:** three ways of handing the lookup tables back (self-extraction, response prefill, cross-model) all fail — extraction is 100% correct on both model sizes, so missing knowledge was never the bottleneck. **Repair 2.0:** showing a small model its own code during repair collapses exploration entirely ("anchoring"), while a fresh-regeneration white-box loop **converges 12/12 on all three de-scaffolded rungs with the 7B — including the business-prose rung nothing else cracked**.

1 The question and the setup

Small, locally-hosted language models are attractive: no data leaves the laptop, no per-token bill, and an 8 GB Apple Silicon MacBook can host a 3B model comfortably. The open question is whether such a model can produce *numerically correct* business logic — not plausible-looking code, but code whose output matches ground truth to the cent, on every row.

The task is fixed throughout. A model must emit a Python function `process_calculations(csv_path)` that applies a ten-rule pricing and commission ruleset to each row of a CSV and returns the running total `total_receivables`. Success is **12/12**: all twelve ground-truth rows matched within a 0.5 tolerance. The model is fixed at Qwen2.5-Coder-3B-Instruct-4bit (the *Coder* variant, via `mlx-lm`); we never swap in a larger or different model. Every experiment holds the model, the harness, and the arithmetic constant, and changes only the wording of the spec.

2 The initial win — and why the old solution failed

A five-rule version of the spec, each rule a single unambiguous operation, is solved by the 3B on the first try: **12/12, no repairs**. The trouble begins at ten rules. Running the article's original *True/False* spec, the model's

best attempts are **3/12 on the 3B** and **1/12 on the 7B** — the architecture is right but every row is 2–5% off. Two failures, both *linguistic* rather than a capability ceiling, explain it:

- **Rule 4 — the volume tier.** The spec's three-way tier ($<100 \rightarrow 0\%$, $<1000 \rightarrow 5\%$, $\geq 1000 \rightarrow 10\%$) is collapsed by the model into `0.05 if v<100 else (0.1 if v<1000 else 0.1)` — wrong in two of three branches.
- **Rule 10 — the boolean.** The `tax verified` cell is the string `"False"`, which is *truthy* in Python, so `not row["tax verified"]` skips withholding at exactly the rows where it should apply.

The fix (the "v2" spec) re-expresses the *same ten rules* in the model's dialect: Rule 4 becomes two independent additive checks (`if v>=100: +0.05; if v>=1000: +0.05`) with an explicit "no elif"; the boolean becomes `yes / no` text in both spec and data; rates are handed over as literal Python dictionaries and bare decimals. The arithmetic is bit-identical to the original (maximum absolute difference in targets = 0.0). The result: **12/12 on the first greedy attempt**. Same problem, phrased so the model cannot trip — the lever is the spec, not the model.

3 The sweet-spot experiment

The v2 spec passes and the original fails, but those are two distant points. *Where exactly* is the boundary, and how technical must a spec author be to stay on the right side of it? To answer precisely we build a ladder of eight specs. Every rung describes the identical ten-rule problem against identical targets; the harness is frozen at the configuration that produced the validated 12/12 (same instruction wrapper, same decoding, same scorer). **Only the spec file changes**, so any change in outcome is attributable to the spec's dialect alone.

Rungs L1–L5 strip one layer of authoring scaffolding at a time, from the heavily-engineered v2 dialect down to the natural original. Rungs L6–L8 are *controls*: they keep the literal lookup tables but re-arm the volume-tier and boolean "traps," to test whether those traps are what actually break the model.

Greedy first: the deterministic result

We score each spec two ways. The simplest is to run it once with **greedy** decoding (temperature 0), which always takes the single most-likely next token — so the *same prompt yields the identical program every time*, giving one reproducible score per spec (every spec carries an explicit one-line normalization step so letter-case never confounds the reading — see the methodology note in §4). Greedy already draws the boundary: **every rung that hands over the literal lookup tables passes 12/12** (L1, L6, L7, L8), while every rung that removes them fails (L3–L5). The one exception is **L2** — it keeps the tables but phrases the volume rule as terse additive prose, which greedy mis-formulates (see §4).

Rung	Spec dialect	Greedy score	Result
L1	Prescriptive pseudocode	12/12	PASS
L2	Declarative spec	9/12	fail
L3	Business prose	3/12	fail
L4	Natural rules (tier)	3/12	fail
L5	Original dialect (boolean)	3/12	fail
L6	Tables + natural tier	12/12	PASS
L7	Tables + True/False	12/12	PASS
L8	Tables + tier + True/False	12/12	PASS

Table 1 · Greedy (deterministic) — one temperature-0 attempt per spec. Tables present $\Rightarrow$ 12/12 (except L2's prose-additive volume rule); tables removed (L3–L5) $\Rightarrow$ fail. A single deterministic shot already reproduces the boundary the sampled view confirms below.

Sampling: measuring reliability

A single greedy shot is still a knife-edge — it carries whatever the top completion produces, and a trivial rewording can flip it (a one-line change to a spec's *title* once flipped L1 from 12/12 to 0/12). To gauge how *reliably* a spec works, we also **sample**: at temperature 0.4 the same prompt yields *different* code on each run, so we ask each spec ten times and record the **best of the ten** (reachable?) and the **pass-rate** (how many of the ten were fully correct). The sampled staircase below confirms the same boundary and adds the reliability dimension a single greedy shot cannot show.

Figure 1 (next page) shows all eight specs as the model sees them, scaled to a single page. The prose after it walks through each rung.

Figure 1 · The eight-rung spec ladder (exactly the text fed to the model, HTML authoring notes stripped)

L1 · Prescriptive pseudocode <pre># Pricing & commission calculation spec Build `process_calculations(csv_path)` that reads a CSV and returns a list of dicts, one per row, each containing the key `total_receivables`. ## Reading the CSV Normalize every row before applying the rules - strip surrounding spaces from the headers and **lower-case every text value** so comparisons work regardless of case. Copy this exactly: row = {key.strip(): value.strip().lower() for key, value in row.items()} This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume` and `productid` to numbers, and treat an empty cell as its stated default. ## Input columns - `productid` : "1", "2", or "3" - `status` : "individual" or "corporate" - `volume` : an integer - `season` : "spring", "summer", "winter", or "autumn" - `customer` : "strategic" or "non-strategic" - `region` : "emea" or "non-emea" - `payment type` : "ach/wire" or "credit-card" - `transaction type` : "domestic", "international", or "cross-border" - `product category` : "electronics", "groceries", or "luxury" - `tax verified` : "yes" or "no" (an empty cell counts as "no") ## Lookup tables (copy these exactly) PRODUCT_RATES = { 1: {"individual": 1000, "corporate": 900}, 2: {"individual": 1500, "corporate": 1200}, 3: {"individual": 2000, "corporate": 1700}, } CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5} ## The 10 rules (apply in this exact order) 1. **Rate.** Look up `rate = PRODUCT_RATES[productid][status]`. 2. **Base.** `receivable_before_discount = volume * rate`. 3. **Spring discount rate.** Start `spring_rate = 0.0`. If `season` is "spring", set `spring_rate = 0.05`. 4. **Volume discount rate.** Build it up with two independent checks (do NOT use elif): Start `volume_rate = 0.0`. - If `volume >= 100`, add `0.05` to `volume_rate`. - If `volume >= 1000`, add another `0.05` to `volume_rate`. (So volume under 100 gives 0.0, 100-999 gives 0.05, 1000 or more gives 0.10.) 5. **Strategic discount rate.** Start `strategic_rate = 0.0`. If `customer` is "strategic", set `strategic_rate = 0.05`. 6. **Apply discounts.** `total_discount_rate = spring_rate + volume_rate + strategic_rate`. `receivable_after_discount = receivable_before_discount * (1 - total_discount_rate)`. 7. **Region loading.** Start `region_loading = 0.0`. If `region` is "emea", set `region_loading = 0.10 * receivable_after_discount`. 8. **Payment loading.** Start `payment_loading = 0.0`. If `payment type` is "credit-card", set `payment_loading = 0.025 * receivable_after_discount`. 9. **Commission.** Look up `multiplier = CATEGORY_MULTIPLIER[product category]`. `commission = (region_loading + payment_loading) * multiplier`. 10. **Tax withholding and total.** `subtotal = receivable_after_discount + commission`. Start `withholding = 0.0`. - If `tax verified` is "yes" (or the cell was empty), set `withholding = 0.03 * subtotal`. - If `tax verified` is "yes", leave `withholding = 0.0`. `total_receivables = subtotal - withholding`. Output: a list of dicts, one per input row, each with the key `total_receivables`.</pre>	L4 · Natural rules (tier) <pre># Pricing & commission calculation spec Write `process_calculations(csv_path)` that reads the CSV and returns one dict per row with the key `total_receivables`. ## Reading the CSV Normalize every row before applying the rules - strip surrounding spaces from the headers and **lower-case every text value** so comparisons work regardless of case. Copy this exactly: row = {key.strip(): value.strip().lower() for key, value in row.items()} This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume` and `productid` to numbers, and treat an empty cell as its stated default. ## Input columns - `productid` : "1", "2", or "3" - `status` : "individual" or "corporate" - `volume` : an integer - `season` : "spring", "summer", "winter", or "autumn" - `customer` : "strategic" or "non-strategic" - `region` : "emea" or "non-emea" - `payment type` : "ach/wire" or "credit-card" - `transaction type` : "domestic", "international", or "cross-border" - `product category` : "electronics", "groceries", or "luxury" - `tax verified` : "yes" or "no" (an empty cell counts as "no") ## The calculation The per-unit price depends on `productid` and `status`. For productid 1 it is \$1,000 when status is individual and \$900 when corporate; for productid 2 it is \$1,500 / \$1,200; for productid 3 it is \$2,000 / \$1,700. The base receivable is `volume` times that per-unit price. Then three discounts come off, and they stack (add together): - **5%** if the `season` is spring; a volume discount based on how much was ordered: under 100 units gets no volume discount, 100 to 999 units gets 5%, and 1000 units or more gets 10%; - **5%** if the `customer` is strategic. Apply the combined discount to get the receivable after discount: Two surcharges are then figured from the receivable after discount: - **10%** if the `region` is EMEA; - **2.5%** if the `payment type` is credit-card. Commission is the sum of those two surcharge amounts, scaled by a factor that depends on `product category`: electronics x1.2, groceries x0.8, luxury x1.5. The subtotal is the receivable after discount plus the commission. If `tax verified` is "no" (or blank), withhold **3%** of the subtotal; if it is "yes", withhold nothing. `total_receivables` is the subtotal minus the withholding.</pre>
L2 · Declarative spec <pre># Pricing & commission calculation spec `process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the key `total_receivables`. ## Reading the CSV Normalize every row before applying the rules - strip surrounding spaces from the headers and **lower-case every text value** so comparisons work regardless of case. Copy this exactly: row = {key.strip(): value.strip().lower() for key, value in row.items()} This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume` and `productid` to numbers, and treat an empty cell as its stated default. ## Input columns - `productid` : "1", "2", or "3" - `status` : "individual" or "corporate" - `volume` : an integer - `season` : "spring", "summer", "winter", or "autumn" - `customer` : "strategic" or "non-strategic" - `region` : "emea" or "non-emea" - `payment type` : "ach/wire" or "credit-card" - `transaction type` : "domestic", "international", or "cross-border" - `product category` : "electronics", "groceries", or "luxury" - `tax verified` : "yes" or "no" (an empty cell counts as "no") ## Lookup tables ... PRODUCT_RATES = { 1: {"individual": 1000, "corporate": 900}, 2: {"individual": 1500, "corporate": 1200}, 3: {"individual": 2000, "corporate": 1700}, } CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5} ## The calculation The base receivable is `volume * PRODUCT_RATES[productid][status]`. Three discount rates then apply, and they add together: - a spring rate of `0.05` when `season` is "spring", otherwise `0.0`; - a volume rate that is `0.05` for `volume >= 100` plus an additional `0.05` for `volume >= 1000` (the two thresholds are independent and stack, so `volume >= 1000` earns the full `0.10`); - a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`. The receivable after discount is the base times `(1 - sum of those three rates)`. Two loadings are computed from the receivable after discount: - a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`; - a payment loading of `0.025 * receivable_after_discount` when `payment type` is "credit-card", else `0.0`. The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`. The subtotal is `receivable_after_discount + commission`. Tax withholding is `0.03 * subtotal` when `tax verified` is "no" (or empty), and `0.0` when it is "yes". `total_receivables = subtotal - withholding`.</pre>	L5 · Original dialect (boolean) <pre># Pricing & commission calculation spec Write `process_calculations(csv_path)` that reads the CSV and returns one dict per row with the key `total_receivables`. ## Reading the CSV Normalize every row before applying the rules - strip surrounding spaces from the headers and **lower-case every text value** so comparisons work regardless of case. Copy this exactly: row = {key.strip(): value.strip().lower() for key, value in row.items()} This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume` and `productid` to numbers, and treat an empty cell as its stated default. ## Input columns - `productid` : "1", "2", or "3" - `status` : "individual" or "corporate" - `volume` : an integer - `season` : "spring", "summer", "winter", or "autumn" - `customer` : "strategic" or "non-strategic" - `region` : "emea" or "non-emea" - `payment type` : "ach/wire" or "credit-card" - `transaction type` : "domestic", "international", or "cross-border" - `product category` : "electronics", "groceries", or "luxury" - `tax verified` : True or False (an empty cell counts as False) ## The calculation The per-unit price depends on `productid` and `status`. For productid 1 it is \$1,000 when status is individual and \$900 when corporate; for productid 2 it is \$1,500 / \$1,200; for productid 3 it is \$2,000 / \$1,700. The base receivable is `volume` times that per-unit price. Then three discounts come off, and they stack (add together): - **5%** if the `season` is spring; a volume discount based on how much was ordered: under 100 units gets no volume discount, 100 to 999 units gets 5%, and 1000 units or more gets 10%; - **5%** if the `customer` is strategic. Apply the combined discount to get the receivable after discount: Two surcharges are then figured from the receivable after discount: - **10%** if the `region` is EMEA; - **2.5%** if the `payment type` is credit-card. Commission is the sum of those two surcharge amounts, scaled by a factor that depends on `product category`: electronics x1.2, groceries x0.8, luxury x1.5. The subtotal is the receivable after discount plus the commission. The `tax verified` column is True or False; when the sale has not been tax-verified, withhold 3% of the subtotal; when it has been verified, withhold nothing. `total_receivables` is the subtotal minus the withholding.</pre>
L3 · Business prose <pre># Pricing & commission calculation spec Write `process_calculations(csv_path)` that reads the CSV and returns one dict per row with the key `total_receivables`. ## Reading the CSV Normalize every row before applying the rules - strip surrounding spaces from the headers and **lower-case every text value** so comparisons work regardless of case. Copy this exactly: row = {key.strip(): value.strip().lower() for key, value in row.items()} This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume` and `productid` to numbers, and treat an empty cell as its stated default. ## Input columns - `productid` : "1", "2", or "3" - `status` : "individual" or "corporate" - `volume` : an integer - `season` : "spring", "summer", "winter", or "autumn" - `customer` : "strategic" or "non-strategic" - `region` : "emea" or "non-emea" - `payment type` : "ach/wire" or "credit-card" - `transaction type` : "domestic", "international", or "cross-border" - `product category` : "electronics", "groceries", or "luxury" - `tax verified` : "yes" or "no" (an empty cell counts as "no") ## The calculation The per-unit price depends on `productid` and `status`. For productid 1 it is \$1,000 when status is individual and \$900 when corporate; for productid 2 it is \$1,500 / \$1,200; for productid 3 it is \$2,000 / \$1,700. The base receivable is `volume` times that per-unit price. Then three discounts come off, and they stack (add together): - **5%** if the `season` is spring; a volume discount of **5%** once `volume` reaches 100, plus a further **5%** once `volume` reaches 1000 (so 100-999 is 5% and 1000-or-more is 10%); - **5%** if the `customer` is strategic. Apply the combined discount to get the receivable after discount: Two surcharges are then figured from the receivable after discount: - **10%** if the `region` is EMEA; - **2.5%** if the `payment type` is credit-card. Commission is the sum of those two surcharge amounts, scaled by a factor that depends on `product category`: electronics x1.2, groceries x0.8, luxury x1.5. The subtotal is the receivable after discount plus the commission. If `tax verified` is "no" (or blank), withhold **3%** of the subtotal; if it is "yes", withhold nothing. `total_receivables` is the subtotal minus the withholding.</pre>	L6 · Tables + natural tier <pre># Pricing & commission calculation spec `process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the key `total_receivables`. ## Reading the CSV Normalize every row before applying the rules - strip surrounding spaces from the headers and **lower-case every text value** so comparisons work regardless of case. Copy this exactly: row = {key.strip(): value.strip().lower() for key, value in row.items()} This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume` and `productid` to numbers, and treat an empty cell as its stated default. ## Input columns - `productid` : "1", "2", or "3" - `status` : "individual" or "corporate" - `volume` : an integer - `season` : "spring", "summer", "winter", or "autumn" - `customer` : "strategic" or "non-strategic" - `region` : "emea" or "non-emea" - `payment type` : "ach/wire" or "credit-card" - `transaction type` : "domestic", "international", or "cross-border" - `product category` : "electronics", "groceries", or "luxury" - `tax verified` : "yes" or "no" (an empty cell counts as "no") ## Lookup tables PRODUCT_RATES = { 1: {"individual": 1000, "corporate": 900}, 2: {"individual": 1500, "corporate": 1200}, 3: {"individual": 2000, "corporate": 1700}, } CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5} ## The calculation The base receivable is `volume * PRODUCT_RATES[productid][status]`. Three discount rates then apply, and they add together: - a spring rate of `0.05` when `season` is "spring", otherwise `0.0`; - a volume rate based on how much was ordered: `0.0` under 100 units, `0.05` from 100 to 999 units, and `0.10` at 1000 units or more; - a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`. The receivable after discount is the base times `(1 - sum of those three rates)`. Two loadings are computed from the receivable after discount: - a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`; - a payment loading of `0.025 * receivable_after_discount` when `payment type` is "credit-card", else `0.0`. The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`. The subtotal is `receivable_after_discount + commission`. Tax withholding is `0.03 * subtotal` when `tax verified` is "no" (or empty), and `0.0` when it is "yes". `total_receivables = subtotal - withholding`.</pre>

L7 · Tables + True/False

```
# Pricing & commission calculation spec
`process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the
key `total_receivables`.
## Reading the CSV
Normalize every row before applying the rules – strip surrounding spaces from the headers
and **lower-case every text value** so comparisons work regardless of case. Copy this exactly:
...
row = {key.strip(): value.strip().lower() for key, value in row.items()}

This turns e.g. 'EMEA' into 'emea' and 'non-EMEA' into 'non-emea'. Then convert 'volume'
and 'productid' to numbers, and treat an empty cell as its stated default.
## Input columns
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : True or False (an empty cell counts as False)
## Lookup tables
...
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}

## The calculation
The base receivable is `volume * PRODUCT_RATES[productid][status]`.
Three discount rates then apply, and they add together:
- a spring rate of `0.05` when `season` is "spring", otherwise `0.0`;
- a volume rate that is `0.05` for `volume >= 100` plus an additional `0.05` for
  `volume >= 1000` (the two thresholds are independent and stack, so `volume >= 1000`
  earns the full `0.10`);
- a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`.
The receivable after discount is the base times `(1 - sum of those three rates)`.
Two loadings are computed from the receivable after discount:
- a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`;
- a payment loading of `0.025 * receivable_after_discount` when `payment type` is
  "credit-card", else `0.0`.
The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`.
The subtotal is `receivable_after_discount + commission`. The `tax verified` column is
True or False; withhold `0.03 * subtotal` when the sale has not been tax-verified, and
`0.0` when it has been verified.
`total_receivables = subtotal - withholding`.
```

L8 · Tables + tier + True/False

```
# Pricing & commission calculation spec
`process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the
key `total_receivables`.
## Reading the CSV
Normalize every row before applying the rules – strip surrounding spaces from the headers
and **lower-case every text value** so comparisons work regardless of case. Copy this exactly:
...
row = {key.strip(): value.strip().lower() for key, value in row.items()}

This turns e.g. 'EMEA' into 'emea' and 'non-EMEA' into 'non-emea'. Then convert 'volume'
and 'productid' to numbers, and treat an empty cell as its stated default.
## Input columns
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : True or False (an empty cell counts as False)
## Lookup tables
...
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}

## The calculation
The base receivable is `volume * PRODUCT_RATES[productid][status]`.
Three discount rates then apply, and they add together:
- a spring rate of `0.05` when `season` is "spring", otherwise `0.0`;
- a volume rate based on how much was ordered: `0.0` under 100 units, `0.05` from 100 to
  999 units, and `0.10` at 1000 units or more;
- a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`.
The receivable after discount is the base times `(1 - sum of those three rates)`.
Two loadings are computed from the receivable after discount:
- a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`;
- a payment loading of `0.025 * receivable_after_discount` when `payment type` is
  "credit-card", else `0.0`.
The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`.
The subtotal is `receivable_after_discount + commission`. The `tax verified` column is
True or False; withhold `0.03 * subtotal` when the sale has not been tax-verified, and
`0.0` when it has been verified.
`total_receivables = subtotal - withholding`.
```

Walking the ladder

- **L1 — Prescriptive pseudocode** (mirrors v2). Literal dicts, additive Rule 4 with "no elif", `yes/no` boolean, bare decimals, step-by-step recipe. The anchor: **reachable, 12/12**.
- **L2 — Declarative spec**. Drops the imperative `Start x = 0.0 ... do NOT use elif` recipe; states *what* each rule computes in plain prose. Tables still handed over. **Reachable, 12/12** — the recipe was not load-bearing.
- **L3 — Business prose**. Removes the literal dictionaries and bare decimals; rates now appear as a `$ / %` narrative the model must turn into its own data structures. **Never reaches more than 4/12**. This is the cliff.
- **L4 — Natural rules**. Re-arms the volume tier as natural bands ("under 100 ... 100–999 ... 1000+"). Already past the cliff: **best 3/12**.
- **L5 — Original dialect**. Also re-arms the True/False boolean — the article's original phrasing. **Best 6/12** (the case fix corrects its region rows, but the commission chain still breaks).
- **L6 — Tables + natural tier**. Control: the L4 tier phrasing, but with the literal tables kept. **Reachable, 12/12**.
- **L7 — Tables + True/False**. Control: the L5 boolean, tables kept. **Reachable, 12/12**.
- **L8 — Tables + tier + True/False**. Control: *both* traps at once, tables kept. Still **reachable, 12/12**.

Rung	Spec dialect	Pass-rate	Best of 10	Reachable
L1	Prescriptive pseudocode	7/10	12/12	✓
L2	Declarative spec	5/10	12/12	✓
L3	Business prose	0/10	4/12	✗
L4	Natural rules (tier)	0/10	3/12	✗
L5	Original dialect (boolean)	0/10	6/12	✗
L6	Tables + natural tier	8/10	12/12	✓
L7	Tables + True/False	1/10	12/12	✓
L8	Tables + tier + True/False	2/10	12/12	✓

Table 2 · Best-of-ten (sampled, temperature 0.4) per rung. Pass-rate is how many of ten attempts hit a full 12/12; "reachable" means at least one did. Tables present (L1, L2, L6, L7, L8) $\Rightarrow$ reachable; tables removed (L3, L4, L5) $\Rightarrow$ not.

4 What the ladder shows — a summary

The break is the **L2 $\rightarrow$ L3 boundary**, and it is sharper than the original two-trap story. Dropping the step-by-step recipe (L1 $\rightarrow$ L2) costs nothing. Dropping the literal lookup tables (L2 $\rightarrow$ L3) is the cliff. And the failures are *not* the tier or boolean at all: at L3–L5 the model handles both correctly in prose. Inspecting *which rows pass* at the failing rungs, it is always exactly the four rows that are `non-emea` **and** `ach/wire` — the rows whose region and payment loadings are both zero, i.e. **commission = 0**. Every wrong row has a non-zero loading, and the error is precisely the missing commission: for instance an electronics/EMEA row scores a ratio of $0.8929 = 1/1.12$, exactly the effect of dropping `commission = (region + payment loading)  $\times$  1.2` from the subtotal.

The load-bearing scaffold is the literal lookup tables, not the phrasing of the famous traps. With the tables present, every dialect is reachable (L1, L2, L6, L7, L8 $\rightarrow$ 12/12); with the tables removed, none is (L3, L4, L5). The tables are necessary and nearly sufficient; the tier and boolean traps are neither necessary nor

sufficient — even both together (L8) still pass. Once the model must build its own data structures from prose, what collapses is the multi-step *commission* chain, not the two single-line constructs. **Greedy decoding draws the same line**: every table-keeping rung passes a single deterministic shot (L1, L6, L7, L8), and every tables-removed rung fails it.

One nuance — phrasing still affects one-shot reliability. L2 keeps the tables and is reachable when sampled (12/12), yet it is the only table-keeping rung to miss on a single greedy shot (9/12): its terse additive volume wording leads greedy to a wrong closed form that drops the sub-100 floor ($0.05 \cdot (1 + (v \geq 1000))$). L6, which states the same rule as explicit tier bands, passes greedy on the first try. So once the tables are present, whether a rule lands on the *first* attempt still depends on stating each case explicitly — an additive one-liner is reachable but not one-shot-reliable.

Takeaway for a spec author. You may write the ten-rule spec in plain declarative prose — but you must (a) hand over the lookup tables verbatim and (b) name every multi-step derived quantity (here, the commission) as its own explicit sub-step. Narrating a multi-step formula in one breath is the thing a 3B cannot reliably reconstruct.

Three methodology notes. First, *single greedy decoding is a knife-edge*: a one-line change to a spec's title flipped L1's deterministic output from 12/12 to 0/12, so a small model must be evaluated by sampling, not one greedy roll. Second, *the instruction wrapper can dominate the result*: an ambiguous "strip spaces from headers" made the model delete the space inside `payment type` and crash, and "never crash" induced `try/except` that swallowed the real bug — both were removed so the wrapper is precise and identical for every rung. Third, *letter-case must be normalized explicitly*: the dataset stores `region` as EMEA / non-EMEA in uppercase, so every spec now carries a one-line `value.strip().lower()` normalization; without it the model compared `region == "emea"` against "EMEA" and silently dropped the EMEA loading on those rows.

5 Can a repair loop rescue the failing rungs?

The repository's standard loop generates code, scores it, and feeds the failure back for another attempt. Its feedback is a **scalar total diff** ("row 3: got X, expected Y") — but the L3 failure is a *localizable structural omission*: the whole commission chain is dropped. A single wrong number cannot say *which* of ten steps is missing, so the model flails. We therefore built `--repair-rules`, a **white-box loop**: a reference computes every intermediate in the pipeline, and on each failing row the loop reports the *first* step that diverges — e.g. "row 0: correct through `payment_loading`, but `commission` = 0.00, should be 3206.25" — asking the model to expose those intermediates so they can be compared.

We ran it on the three failing rungs (three trajectories, up to six iterations) as an A/B on the repair temperature. The localization works — the model does fix the flagged step. At the default temperature (0.6) the score paths *oscillate* (`[4, 0, 1, 0, 0, 9]`): fixing one step regresses another, and nothing converges. **Dropping the repair temperature to 0.3** makes the edits conservative enough to stick.

Rung	Spec dialect	best-of-10 (no repair)	repair-rules @ temp 0.6	repair-rules @ temp 0.3
L3	Business prose	4/12	8/12 (0/3)	7/12 (0/3)
L4	Natural rules (tier)	3/12	4/12 (0/3)	8/12 (0/3)
L5	Original dialect (boolean)	6/12	9/12 (0/3)	12/12 (1/3, iter 2)

Table 3 · White-box repair on the failing rungs. Cell shows best score and, in parentheses, trajectories (of three) that reached a full 12/12. At temperature 0.3 the boolean rung **L5 converges to 12/12 in a single repair iteration** (path `[4, 12]`); L4 climbs to 8/12; L3 still resists.

Interpretation. White-box, per-step feedback is a *real* lever — enough to rescue the boolean rung and nearly the tier rung at a low temperature — but **not a guarantee** for a fully de-scaffolded spec. Rebuilding the entire commission chain from pure prose (L3) remains out of reach in this budget, and the extra burden of emitting a dozen intermediate values every turn itself destabilizes a 3B's generation (the residual zero-scores are crashes). The dependable fix for L3 is still the *spec* — put the table back, or run a spec-repair loop that rewrites the prose toward the L1/L2 dialect — rather than out-arguing the model with cleverer diffs.

6 Round 2 — scale, hybrids, and a second-generation repair loop

Round one ended with a single stubborn cell: L3, the fully de-scaffolded business prose. The second round asks three questions about it. *Does the cliff move with scale?* — run the identical ladder on `Qwen2.5-Coder-7B-Instruct-4bit`, changing nothing but `--model`. *Is missing knowledge the bottleneck?* — hand the lookup tables back in three different ways. *Can a better loop close it?* — rebuild the repair loop around what round one taught. Every run below re-scores offline from its saved artifact (`prove_ladder.py --tag ...`).

6.1 Scale: the cliff does not move — reachability does

Under greedy decoding the 7B draws *exactly the same boundary* as the 3B: every structured rung passes (and L2, the 3B's one greedy wobble, is repaired by scale to 12/12), while every tables-removed rung fails at 4/12 — one row better than the 3B's 3/12, from case-handling rather than from any commission progress. Four billion additional parameters do not buy a first-try pass on business prose. Sampling is a different story: the 7B reaches 12/12 on L3, L4 and L5 within ten attempts — rungs the 3B never reached in any experiment — but only at a pass-rate of 1–2 in 10. Spec structure is still worth more than parameters: the structured rungs run at 8–10 in 10.

Rung	Spec dialect	3B greedy	7B greedy	3B best-of-10	7B best-of-10 (pass-rate)
L1	Prescriptive pseudocode	12/12	12/12	12/12	12/12 (10/10)
L2	Declarative spec	9/12	12/12	12/12	12/12 (8/10)
L3	Business prose	3/12	4/12	4/12	12/12 (2/10)
L4	Natural rules (tier)	3/12	4/12	3/12	12/12 (1/10)
L5	Original dialect (boolean)	3/12	4/12	6/12	12/12 (2/10)
L6	Tables + natural tier	12/12	10/12	12/12	12/12 (1/10)
L7	Tables + True/False	12/12	12/12	12/12	12/12 (9/10)
L8	Tables + tier + True/False	12/12	10/12	12/12	12/12 (1/10)

Table 4 · The identical ladder at both model sizes (row colour = 7B greedy result). The greedy cliff is unchanged; sampling makes every rung reachable for the 7B, at 1–2/10 reliability on the prose rungs.

Scale swaps traps rather than removing them. On the natural-tier control rungs the 7B gets stuck at exactly 10/12 in 17 of 20 samples (L6 and L8, greedy included): it writes `0.05 if volume >= 100 else 0.10 if volume >= 1000 else 0.0` — a misordered chained ternary whose `0.10` branch is unreachable, so every `volume ≥ 1000` row gets half its discount. This is the article's original Rule-4 tier trap resurfacing in a new surface form: the 3B collapsed the tier bounds, the 7B misorders the ternary — and the additive phrasing (L7, kept from the v2 dialect) stays robust at both sizes.

6.2 Hybrids: the model knows the tables — it cannot wield them from prose

If the cliff were about missing data, handing the tables back should fix it. Three probes hand them back three ways. **Self-extraction:** a first pass asks the model to rebuild `PRODUCT_RATES / CATEGORY_MULTIPLIER` from the

prose as Python literals; a second pass codes with its own extraction appended. **Response prefill:** the reply is *seeded* so it already opens with the true tables written out as code, and the model merely continues. **Cross-model:** the 7B extracts, the 3B codes.

Rung	Spec dialect	plain best-of-10	self-extract best-of-10	prefill best-of-10	7B extracts, 3B codes (greedy)
L3	Business prose	4/12	0/12 tables ✓	4/12	4/12 tables ✓
L4	Natural rules (tier)	3/12	6/12 tables ✓	4/12	1/12 tables ✓
L5	Original dialect (boolean)	6/12	9/12 tables ✓	2/12	0/12 tables ✓

Table 5 · Handing the tables back to the failing rungs (3B coder; best scores). "tables ✓" = the extracted constants matched ground truth exactly — which they did in **every run, on both model sizes**.

Extraction never failed once. What fails, with correct tables in hand — its own! — is the same assembly: the coder reads "a **10%** surcharge" as `receivable *= 1.10` instead of materializing a separate `0.10 × receivable amount`, so the two loading values never exist as variables and the commission has nothing to sum. (A second failure seam appears only in the two-stage pipeline: the extractor writes "1" -style string keys, and half the coder samples index them with `int` — the model is not even consistent with itself across two of its own turns.) This sharpens round one's conclusion: the load-bearing scaffold is not the table *values* but the **code-shaped decomposition** that comes with them — bare decimals and named quantities that force each step to exist.

6.3 Repair 2.0: anchoring versus regeneration

Round one's white-box loop regenerated from scratch each iteration. Repair 2.0 added four levers on top of the per-step diffs: the *previous code in the prompt* with "edit only the first wrong step", a freeze-list of steps verified correct on every row, *branch-3* (three candidates per iteration, keep the best), and a temperature anneal. The first surprise was that the full combination is **worse than round one** — and the per-iteration logs isolate the culprit cleanly. In the **anchored** runs the instrumentation succeeds (all thirteen intermediates exposed), the localization works (culprit named, freeze-list built) — and then every one of twelve repair candidates per level scores an *identical* 3/12 at every temperature from 0.5 to 0.2. Shown its own code, a 3B copies it. Removing the code from the prompt (**fresh**) restores exploration instantly: candidate scores per iteration jump to spreads like `[12, 2, 0]`, and the loop starts climbing.

Repair-2.0 variant	L3 business prose	L4 natural tier	L5 original dialect
anchored — previous code in the prompt (3B)	3/12	3/12	3/12
fresh — regenerate from feedback (3B)	4/12	9/12	12/12 ✓ iter 4
fresh, cool-start control 0.3→0.15 (3B)	4/12	3/12	3/12
fresh (7B)	12/12 ✓ iter 5	12/12 ✓ iter 2	12/12 ✓ iter 5

Table 6 · Repair 2.0 variants on the failing rungs (best score; ✓ = converged to 12/12 at the given iteration). Anchoring flat-lines despite working localization; fresh regeneration rescues L5 on the 3B; **the 7B converges on all three rungs**.

Two honest footnotes from the controls. The cool-start run *refuted* our own working hypothesis — the hot anneal phase (0.6→0.4, scores 0–2 everywhere) looked like wasted budget, yet starting cold did not reproduce the L5 rescue: at 3B scale these single-trajectory outcomes are a lottery over candidates, and what moves the odds is candidate *volume* (branch × iterations × trajectories), not schedule fine-tuning. And nearly every scoring iteration picks one good candidate out of `[x, 0, 0]` — branch-3 is three lottery tickets per iteration, which is precisely why it helps.

The round-2 headline: 7B + fresh white-box repair closes the board. All three de-scaffolded rungs converge to 12/12 within five iterations — including L3, the business-prose cell that resisted every other lever in both rounds. Business prose in, numerically-correct code out, fully on-device.

6.4 The revised recipe

For this ten-rule task on a small local model, in increasing order of cost: **(1)** write the spec in the model's dialect — literal tables, bare decimals, named steps — and the 3B passes greedy, first try; **(2)** keep the business prose and run **7B + fresh white-box repair** — converged 12/12 on every prose rung in ≤ 5 iterations in our runs; **(3)** sample the 7B and accept 1–2-in-10 reliability. What does *not* work: handing the tables back in any form (the knowledge was never missing), showing a small model its own code during repair (anchoring), or expecting scale alone to fix a first-try dialect failure.

7 Conclusion

Yes — a tiny AI on an 8 GB MacBook can write real, numerically-correct business logic, *provided the specification is written in its dialect*. Holding the model fixed and moving only the words, we located the boundary precisely: correctness survives plain declarative prose but collapses the moment the author stops handing over the data tables and expects the model to assemble a multi-step derived quantity from narrative. The two constructs long blamed for the failure turn out to be handled fine; the true fragility is structural composition.

The second round hardened that finding from three directions and then, finally, broke through it. Scale does not move the cliff — the 7B fails greedy business prose exactly where the 3B does, and re-arms the old tier trap in a new disguise. Knowledge is not the constraint — both models rebuild the tables from prose flawlessly and still cannot assemble the chain. And repair loops obey a sharp rule of their own: **shown its own code, a small model stops exploring; forced to regenerate against per-step diffs, it climbs**. The practical frontier today is a two-tier recipe: engineer the spec and a 3B delivers on the first deterministic attempt — or keep the prose, add four billion parameters and a fresh-regeneration white-box loop, and converge in a handful of iterations. Either way the original lesson stands: **the dependable lever is the words the model reads, not the weights it runs on**.

Models: `mlx-community/Qwen2.5-Coder-3B-Instruct-4bit` and (round 2) `...-7B-Instruct-4bit` via `mlx-lm 0.29.1` (Python 3.9, Apple Silicon; round-2 runs on a 16 GB M2 Pro — the 4-bit 7B (~4.3 GB) also fits an 8 GB machine). Dataset: 12 rows, tolerance 0.5. Ladder metric: best-of-10 at temperature 0.4. Repair: iteration-1 greedy, subsequent iterations sampled. All specs, code, datasets, and result JSONs are in `experiments/sweet-spot/`; this report is generated from those artifacts by `docs/build_report.py`.